

3 Parametric Approximation

Learning objectives

After studying this chapter, the reader should be able to:

1. distinguish approximations of a cost-to-go function, Q-factor, policy, and model, and select a suitable regression, classification, or continuous-action output form;
2. construct linear-in-parameter approximators from fixed features and formulate their least-squares and ridge-regularized fits;
3. explain learned feature construction in shallow and deep MLPs, backpropagation and gradient-based parameter updates, and the role of structured neural encoders;
4. explain how the fitting distribution, declared data support, regularization, and target-generation versus fitting error determine what a trained approximator represents; and
5. compare approximators using prediction error, greedy-action agreement, and one-step decision regret, and interpret these metrics in the car-following case.

Chapter roadmap

Chapter 2 identified parametric approximation as one important ingredient of approximate DP: a cost function, Q-factor, policy, or model that cannot be stored exactly is represented by finitely many parameters. This chapter isolates that ingredient before Chapter 4 returns to Bellman- and trajectory-generated targets. We first identify the object and its output structure, then compare fixed-feature linear and learned- feature neural architectures. Data and control-relevant validation lead to a common mobility example. Figure 3.1 shows this progression.

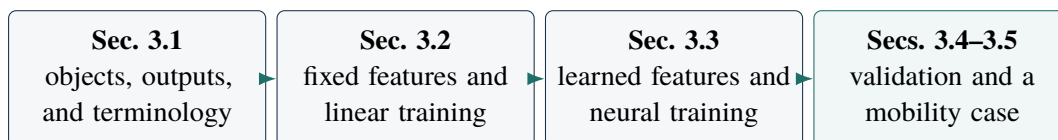


Figure 3.1: Chapter 3 roadmap. The target object, approximation architecture, training rule, and control-facing validation are treated as distinct design layers.

3.1 Parametric Objects and Architecture Choices

3.1.1 From a Target Object to a Finite Parameter Vector

Let ξ denote the input to a target function h . A parametric architecture is a function class

$$\mathcal{H}_p = \{\tilde{h}(\cdot; r) : r \in \mathbb{R}^p\}, \quad \tilde{h}(\xi; r) \approx h(\xi). \quad (3.1)$$

The architecture specifies which input–output maps can be represented. A training rule uses targets to select one parameter vector $\hat{r}$; the algorithm that produced those targets is a separate design choice. This distinction becomes important in Chapter 4, where the same architecture can be trained using model-based Bellman calculations, simulated returns, or real transitions.

The four recurring objects in this book are

$$\tilde{J}(x; r_J) \approx J(x), \quad \tilde{Q}(x, u; r_Q) \approx Q(x, u), \quad (3.2)$$

$$\tilde{\mu}(x; r_\mu) \approx \mu(x), \quad \tilde{f}(x, u; r_f) \approx f_{\text{mean}}(x, u). \quad (3.3)$$

Here J and Q are generic targets. Depending on how their targets are generated, they may represent J^* and Q^* or the policy-specific objects J_μ and Q_μ . The architecture need not be relabeled until that target source matters. This convention keeps the present chapter focused on representation and fitting. For stochastic dynamics, $f_{\text{mean}}(x, u) = \mathbb{E}_{w|x, u}[f(x, u, w)]$ denotes a conditional-mean predictor; learning a complete transition distribution is a different model-learning task developed in Chapter 7. Figure 3.2 separates target generation, fitting, and deployment so that changing one layer does not silently redefine the others. In the figure, the sample index is s , and $\beta^{(s)}$ denotes the target output paired with input $\xi^{(s)}$; ideally $\beta^{(s)} = h(\xi^{(s)})$, although a computed or measured target may also contain approximation or observation error.

Table 3.1 records what each stored object does and what computation still remains at execution.

For a finite-horizon problem, the target objects are generally stage dependent, for example

$$\tilde{J}_k(x; r_{J,k}), \quad \tilde{Q}_k(x, u; r_{Q,k}), \quad k = 0, \dots, N - 1. \quad (3.4)$$

A single map intended to represent genuinely stage-dependent targets generally needs the stage, remaining horizon, reference preview, or other nonstationary information in its input. Otherwise parameter sharing imposes a stationary representation, unless the target is in fact stage invariant.



At execution, the stored map may be evaluated directly
or inserted into an online model-based minimization.

Figure 3.2: Common parametric-approximation workflow. The target generator, the function class, and the fitting procedure can be changed independently.

Table 3.1: What a stored approximation supplies at execution.

Stored object	Execution consequence
Cost function $\tilde{J}(x)$	A model and an online minimization are still needed to compare successor states.
Q-factor $\tilde{Q}(x, u)$	A finite action is selected by $\arg \min_{u \in \mathcal{U}(x)} \tilde{Q}(x, u)$; a continuous action still needs an optimizer or a separate policy approximator.
Policy $\tilde{\mu}(x)$	An action is produced directly, subject to any declared feasibility, saturation, or safety layer.
Model $\tilde{f}(x, u)$	Prediction becomes available, but an optimizer or policy is still required to choose an action.

3.1.2 Regression, Classification, and Policy Outputs

The object determines the required output structure before it determines the internal architecture. Figure 3.3 separates three common cases.

Cost, Q-factor, and model regression.

Cost functions use a scalar output, and vector dynamics use one output per predicted state component. For a finite action set $\mathcal{U} = \{u^{(1)}, \dots, u^{(n_a)}\}$, one convenient Q-factor architecture takes x as its input and returns all action values at once:

$$\tilde{\mathbf{Q}}(x; r_Q) = [\tilde{Q}(x, u^{(1)}; r_Q) \dots \tilde{Q}(x, u^{(n_a)}; r_Q)]^T. \quad (3.5)$$

The alternative scalar architecture takes (x, u) as its input and returns one $\tilde{Q}(x, u; r_Q)$. This second form extends naturally to a continuous action space, but execution then requires a continuous minimization over u or a separate policy approximator. Special continuous-action Q-factor forms that simplify this minimization are introduced in Chapter 6.

When several outputs share the same intermediate feature vector, that common map is often called a *backbone*; the final output-specific mappings are called *heads*. A shared backbone lets observations for one action or state component influence common features. Completely separate approximators do not share that information. These names describe implementation structure, not a new class of control algorithm.

Finite-action policy classification.

When deployment must choose one member of a finite set, its output can be treated as a class label. This guarantees an admissible discrete output and avoids imposing an arbitrary numerical distance between action indices. If the actions possess useful geometry—for example, ordered torque levels—an ordinal or cost-sensitive classifier, or a Q-factor that retains their cost consequences, may use that geometry. The basic classifier assigns one score to each action

and converts the scores into differentiable probabilities, so categorical targets can be fitted by continuous parameter optimization. For $\mathcal{U} = \{u^{(1)}, \dots, u^{(n_a)}\}$, let $z_j(x; r)$ be one score per action and define

$$p_j(x; r) = \frac{\exp z_j(x; r)}{\sum_{\ell=1}^{n_a} \exp z_\ell(x; r)}, \quad (3.6)$$

$$\tilde{\mu}(x; r) = u^{(j^*)}, \quad j^* \in \arg \max_{j \in \{1, \dots, n_a\}} p_j(x; r). \quad (3.7)$$

Sampling from p_j defines a randomized policy; the arg max rule defines a deterministic feedback law. More explicitly, $\pi_r(u^{(j)} | x) = p_j(x; r)$ defines the stochastic policy, while $\tilde{\mu}$ denotes its deterministic arg max deployment. A classifier learns action labels or probabilities directly, whereas a Q-factor estimates action-conditioned costs and is minimized afterward. Classifier losses and policy-target generation are developed in Chapter 6.

Continuous-action policy regression.

A vector-valued approximator can produce the action directly. Its output map should encode known constraints when convenient. For scalar actuation,

$$\tilde{\mu}(x; r_\mu) = u_c + u_s \tanh(z_\mu(x; r_\mu)) \quad (3.8)$$

where u_c is the interval center and $u_s > 0$ is its half-width. Thus the output lies in $[u_c - u_s, u_c + u_s]$. More general actions may be projected onto $\mathcal{U}(x)$, although projection can change the gradients used in policy training and does not by itself certify safety.

AI/RL terminology bridge

Control texts commonly use *cost function*, *Q-factor*, and *feedback law*. When the chosen architecture is a neural network, RL and machine-learning texts often say *value network* or *critic*, *Q-network*, and *actor* or *policy network*. This book uses both vocabularies when making the connection explicit, while retaining J , Q , and μ as the primary control notation. The word *network* identifies an architecture, not whether its targets came from a model or from samples.

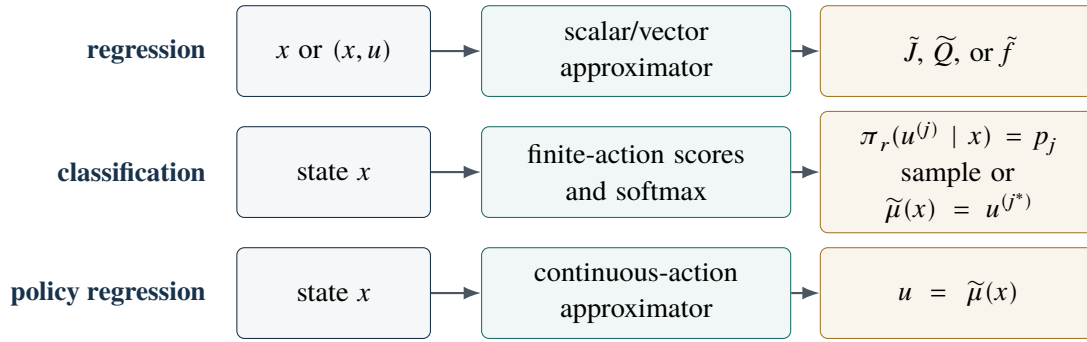


Figure 3.3: Three output forms used by parametric approximators. The internal map may be linear in fixed features or a neural network; the required output semantics do not determine the training algorithm.

3.1.3 Architecture Map

The same output can be represented by different architecture families. Table 3.2 organizes the families by whether the fitted output is linear in its trainable parameters or the feature construction is trained nonlinearly. Within the linear class, polynomial, Fourier, spline, tile, radial-basis, kernel, and physics-derived features differ mainly in support, smoothness, and prior structure. Within the nonlinear class, MLPs learn generic feedforward features, while structured learned encoders exploit spatial, graph, or sequential organization. The table covers the principal families used in this book rather than every possible regressor; decision trees, Gaussian processes, and other supervised-learning models can serve the same input–output role when their computational and uncertainty properties fit the control problem. RBF denotes a radial-basis function.

In this chapter, *neural network* refers to the MLP and structured learned encoder rows. The first two rows instead use features fixed before their output weights are fitted, even though those features can be nonlinear functions of the input.

RBF and kernel architectures are linear in their fitted output weights only when their centers, widths, or kernel locations are fixed before fitting. Learning those feature parameters makes the complete architecture nonlinear in its parameter vector.

Table 3.2: Architecture families used in this chapter.

Parameter dependence	Architecture family	Representation
Linear in fitted weights	Polynomial, Fourier, spline, or structured fixed features	Global smooth terms and physics-informed quantities are selected before fitting; Section 3.2 develops the common linear form.
	Piecewise-constant, tile, RBF, or kernel features	Local indicator or smooth kernel features are placed before fitting; Section 3.2 compares their locality and smoothness.
Nonlinear in the full parameter vector	Shallow or deep MLP	A feedforward neural network learns internal features and the output map jointly; Section 3.3 begins with this baseline.
	Structured learned encoder	CNNs, RNNs, graph neural networks, Transformers, and learned SSMs impose spatial, relational, or sequential structure; Section 3.3 compares them.

3.2 Linear-in-Parameter Architectures and Training

An architecture is *linear in its parameters* when its output is a linear combination of quantities that have already been computed from the input. The input–feature map may itself be highly nonlinear; linearity refers to the unknown coefficients that are fitted. This distinction is valuable because a nonlinear function of the state can still be trained by a convex least-squares problem with an explicit solution. The price is that the feature functions, their number, and their placement are fixed before the coefficients are fitted. Thus Section 3.2.1 asks how to choose the representation, and Section 3.2.2 asks how to fit its weights.

3.2.1 Fixed Feature Constructions

Choose a feature map

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^{n_\phi}, \quad \phi(x) = [\phi_1(x) \quad \cdots \quad \phi_{n_\phi}(x)]^\top. \quad (3.9)$$

A linear-in-parameter cost approximation has the form

$$\tilde{J}(x; r) = \phi(x)^\top r = \sum_{j=1}^{n_\phi} r_j \phi_j(x). \quad (3.10)$$

The architecture is linear in r , not necessarily in x . Polynomial, trigonometric, hinge, radial, and physics-derived nonlinear functions may all appear in ϕ .

Polynomial features.

Polynomial features begin with a constant and the individual state components, then add products of those components. A degree-two map, for example, includes linear terms x_i , squared terms x_i^2 , and interaction terms $x_i x_j$. For the two-state regulation state $x = (e_d, e_v)$, this gives

$$\phi_{\text{quad}}(x) = [1 \quad e_d \quad e_v \quad e_d^2 \quad e_d e_v \quad e_v^2]^\top. \quad (3.11)$$

Higher-degree maps continue in the same way by including products whose total degree does not exceed the selected degree d . Polynomial features are smooth and inexpensive to evaluate. Low-order terms often encode familiar quadratic-control structure, and cross terms represent state interactions. Their support is global, however: changing one coefficient changes the fitted surface over the full domain. High degree also increases the feature count rapidly and can make the fit poorly conditioned unless the state is scaled.

Piecewise-constant or tile features.

For cells $\{\mathcal{C}_j\}_{j=1}^{n_\phi}$,

$$\phi_j(x) = \mathbf{1}\{x \in \mathcal{C}_j\}, \quad \tilde{J}(x; r) = \sum_{j=1}^{n_\phi} r_j \mathbf{1}\{x \in \mathcal{C}_j\}. \quad (3.12)$$

A nonoverlapping partition is a compressed lookup table. Overlapping tilings use multiple partitions whose cell boundaries are shifted relative to one another. A state then activates one tile in each tiling, which can reduce visible dependence on any single boundary. These features are local: changing one weight affects one cell or a small collection of overlapping cells. They are easy to interpret and update, but a single partition is discontinuous at its boundaries and the number of cells grows rapidly with state dimension. Figure 3.4 shows how the indicators in Equation (3.12) are constructed.

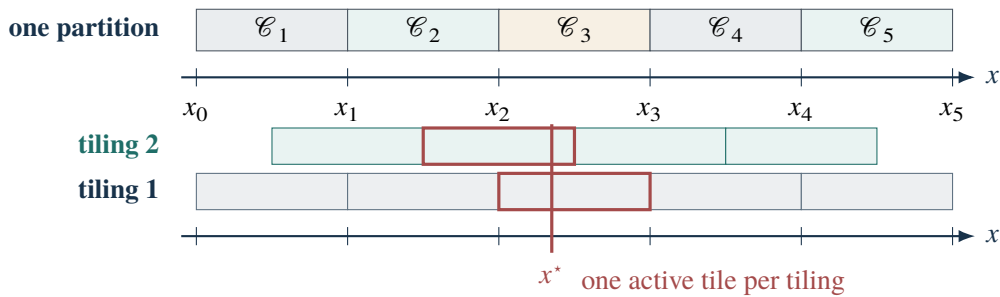


Figure 3.4: Piecewise-constant features on a scalar state. The upper construction uses one nonoverlapping partition. In the lower construction, tiling 2 is shifted relative to tiling 1, and the marked state activates one tile in each tiling.

Radial-basis features.

A smooth local feature centered at c_j is

$$\phi_j(x) = \exp\left(-\frac{\|x - c_j\|^2}{2\sigma_j^2}\right). \quad (3.13)$$

Small σ_j gives local resolution but may leave holes; large σ_j couples more distant states. Far from every center, all RBFs can become small, so adding more centers does not automatically produce reliable extrapolation. RBFs replace the discontinuities of a tile map by smooth local influence and are convenient when nearby states should have similar values. Their center locations and widths nevertheless require a coverage choice, and dense center grids inherit dimensional growth.

Control-informed features.

Known physical or decision structure can be encoded directly. For a spacing constraint $d \geq d_{\min}$,

$$\phi_{\text{risk}}(d) = [d_{\min} - d]_+^2, \quad [z]_+ = \max\{0, z\}. \quad (3.14)$$

This feature makes a one-sided cost shape easier to represent and can reduce the number of generic basis functions required. It does not enforce the constraint: feasibility still belongs to the control problem or to a separate safety mechanism.

The same construction applies to other outputs. A linear Q-factor uses

$$\tilde{Q}(x, u; r_Q) = \psi(x, u)^\top r_Q, \quad (3.15)$$

and a continuous policy can use a vector-valued feature map inside Equation (3.8). Continuous-action Q-factor structures that permit analytic or efficient minimization are introduced with actors and critics in Chapter 6.

Figure 3.5 compares the support and smoothness of these representative fixed features on one normalized scalar state.

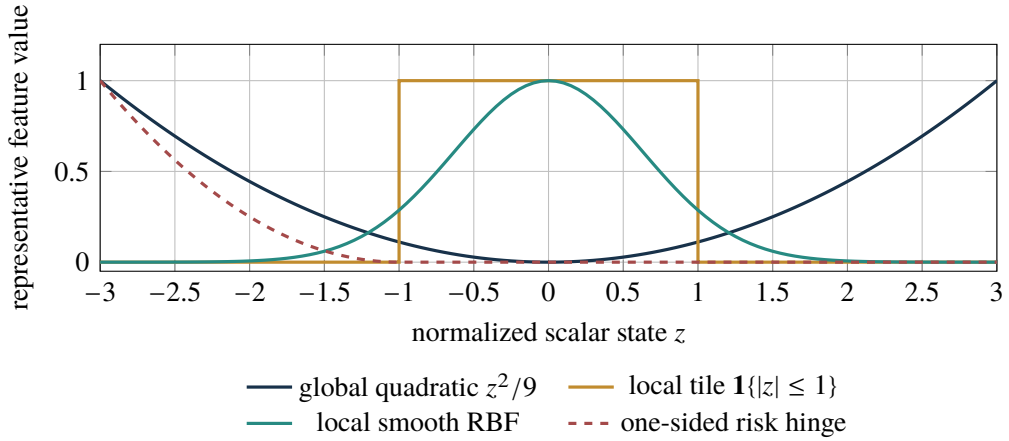


Figure 3.5: Representative terms from Section 3.2.1. A polynomial feature has global support, a piecewise-constant tile has local discontinuous support, an RBF has local smooth support, and a control-informed hinge activates on a declared side of the state space.

3.2.2 Fixed-Target Least Squares

Suppose the target J is available at selected states:

$$\mathcal{D} = \{(x^{(s)}, \beta^{(s)})\}_{s=1}^q, \quad \beta^{(s)} = J(x^{(s)}) + \varepsilon^{(s)}. \quad (3.16)$$

The label error $\varepsilon^{(s)}$ can be random simulation noise, numerical error, or systematic bias. It is not assumed to have zero mean unless that assumption is stated.

Let $\Phi \in \mathbb{R}^{q \times n_\phi}$ have row $\phi(x^{(s)})^\top$ and collect the labels in β . Ordinary least squares solves

$$r_{\text{LS}} \in \arg \min_r \|\Phi r - \beta\|_2^2. \quad (3.17)$$

If Φ has full column rank, the unique solution is

$$r_{\text{LS}} = (\Phi^\top \Phi)^{-1} \Phi^\top \beta. \quad (3.18)$$

This explicit solution makes the advantage of a linear-in-parameter architecture visible. In numerical software one should normally solve the least-squares system using QR or SVD rather than form the inverse in Equation (3.18). Rank deficiency requires a pseudoinverse, additional structure, or regularization.

A three-sample polynomial fit.

For a calculation that can be checked by hand, take the scalar states $x^{(1)} = -1$, $x^{(2)} = 0$, and $x^{(3)} = 1$, the features $\phi(x) = [1, x, x^2]^\top$, and the labels $\beta = [0, 0, 1]^\top$. Then

$$\Phi = \begin{bmatrix} 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad r_{\text{LS}} = \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} \end{bmatrix}^\top, \quad \tilde{J}(x) = \frac{1}{2}x + \frac{1}{2}x^2. \quad (3.19)$$

The number of labels equals the number of linearly independent features, so the polynomial interpolates all three samples. With more samples, the same matrix construction minimizes their aggregate squared residual and need not pass through every point.

With nonnegative sample weights $W = \text{diag}(\omega_1, \dots, \omega_q)$, $\lambda_{\text{reg}} \geq 0$, and a regularization operator L_{reg} , the more general problem is

$$r_\lambda \in \arg \min_r (\Phi r - \beta)^\top W (\Phi r - \beta) + \lambda_{\text{reg}} \|L_{\text{reg}} r\|_2^2, \quad (3.20)$$

with solution

$$r_\lambda = (\Phi^\top W \Phi + \lambda_{\text{reg}} L_{\text{reg}}^\top L_{\text{reg}})^{-1} \Phi^\top W \beta \quad (3.21)$$

when the displayed normal matrix is positive definite. The weights declare which samples matter to the fit; L_{reg} declares which parameter combinations are penalized. The coefficient λ_{reg} sets the tradeoff between matching the labels and keeping the penalized parameter combination small. For $L_{\text{reg}} = \text{diag}(0, 1, \dots, 1)$, the intercept is unpenalized and a larger λ_{reg} shrinks the remaining coefficients. This can improve conditioning and reduce sensitivity to noise, but excessive shrinkage underfits the target. Section 3.4.2 compares these effects visually and discusses other penalties. For very large data sets or feature vectors, incremental and minibatch methods can solve the same objective without assembling a dense normal equation. Those updates are introduced with neural training in Section 3.3.2.

3.3 Nonlinear and Neural Architectures

A fixed-feature architecture may be nonlinear in the state while remaining linear in its fitted coefficients. A neural architecture changes that division of labor: it learns both an internal feature map and the output map that uses those features. The resulting function is generally *nonlinear in the full parameter vector*, and its training problem is usually nonconvex. This additional difficulty buys flexibility because the features need not be selected completely before seeing data. The MLP makes this idea explicit; structured encoders later adapt it to spatial, relational, and sequential mobility data.

3.3.1 Shallow and Multilayer Perceptrons

Let L denote the number of hidden layers in a feedforward neural network. A single-hidden-layer network—called a *shallow MLP* here—has $L = 1$, whereas a deep MLP has $L > 1$ and composes several hidden feature layers. Let $z(x) \in \mathbb{R}^{n_z}$ denote a normalized or otherwise encoded input, set $h_0 = z(x)$, and let $h_\ell \in \mathbb{R}^{m_\ell}$ denote the width- m_ℓ hidden representation. An *activation function* σ_ℓ is a scalar nonlinear map applied componentwise. Then

$$a_\ell = W_\ell h_{\ell-1} + b_\ell, \quad h_\ell = \sigma_\ell(a_\ell), \quad \ell = 1, \dots, L, \quad (3.22)$$

$$\tilde{J}(x; \theta) = r_o^\top h_L + b_o, \quad \theta = \{W_\ell, b_\ell, r_o, b_o\}. \quad (3.23)$$

If every activation were the identity, the composition would collapse to one affine map regardless of depth. Nonlinear activations prevent that collapse. For fixed hidden parameters, $h_L(x)$ is a fixed feature vector and the last layer is linear in (r_o, b_o) . The complete architecture is nonlinear in θ because neural training also moves the hidden weights and biases, thereby changing the feature extractor. This joint adjustment is called *automatic feature construction*. Figure 3.6 shows the resulting separation between learned hidden features and the output head.

For a finite-action Q-factor, the scalar output is replaced by the vector output in Equation (3.5). For a continuous-action Q-factor, the encoded input becomes $z(x, u)$ and the output remains scalar.

Common componentwise activations include

$$\sigma_{\text{ReLU}}(z) = [z]_+, \quad \sigma_{\text{softplus}}(z) = \log(1 + e^z), \quad \sigma_{\text{tanh}}(z) = \tanh z, \quad \sigma_{\text{sig}}(z) = \frac{1}{1 + e^{-z}}. \quad (3.24)$$

The rectified linear unit (ReLU) is inexpensive, piecewise linear, and does not saturate for positive inputs. Its kink is nondifferentiable, and a unit that remains on the negative side can receive no gradient. Softplus is a smooth approximation of ReLU, but loses exact sparsity and is more expensive. The hyperbolic tangent is centered and bounded, which suits normalized signed signals, but its gradient becomes small in saturation. The logistic sigmoid maps to $(0, 1)$ and is natural for probabilities or gates; as a hidden activation it is noncentered and also saturates. Hidden- and output-layer activations serve different purposes: a linear output is natural for an unconstrained cost, whereas a bounded actor or probability output needs a range-compatible map.

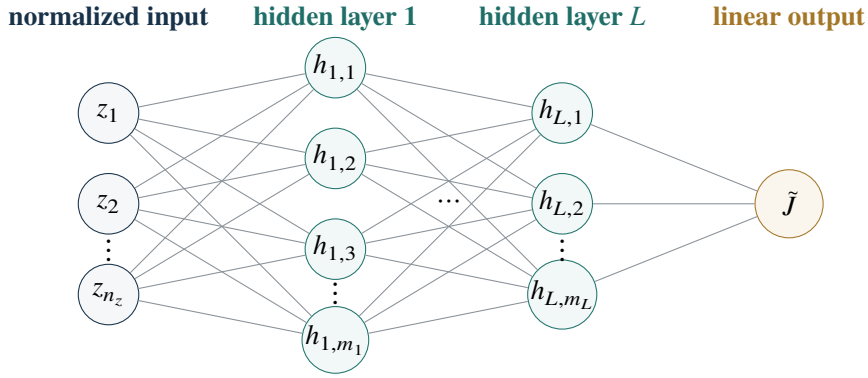


Figure 3.6: MLP for scalar cost-function approximation. Hidden layers construct features jointly with the linear output layer. Changing the input encoding and/or output head gives a Q-factor, policy, or model without changing this basic construction.

A hand-computable shallow ReLU example.

Three ReLU units form the localized triangular feature

$$\psi_{\Delta}(z) = [z + 1]_+ - 2[z]_+ + [z - 1]_+ = \begin{cases} 0, & z \leq -1, \\ z + 1, & -1 < z \leq 0, \\ 1 - z, & 0 < z \leq 1, \\ 0, & z > 1. \end{cases} \quad (3.25)$$

It is a shallow MLP with hidden weights $(1, 1, 1)$, biases $(1, 0, -1)$, output weights $(1, -2, 1)$, and zero output bias. A trained network moves these hinge locations, slopes, and combinations rather than receiving the localized feature in advance. Deeper layers repeat this construction hierarchically. Figure 3.7 illustrates both common activation shapes and the construction of a localized feature from ReLU hinges.

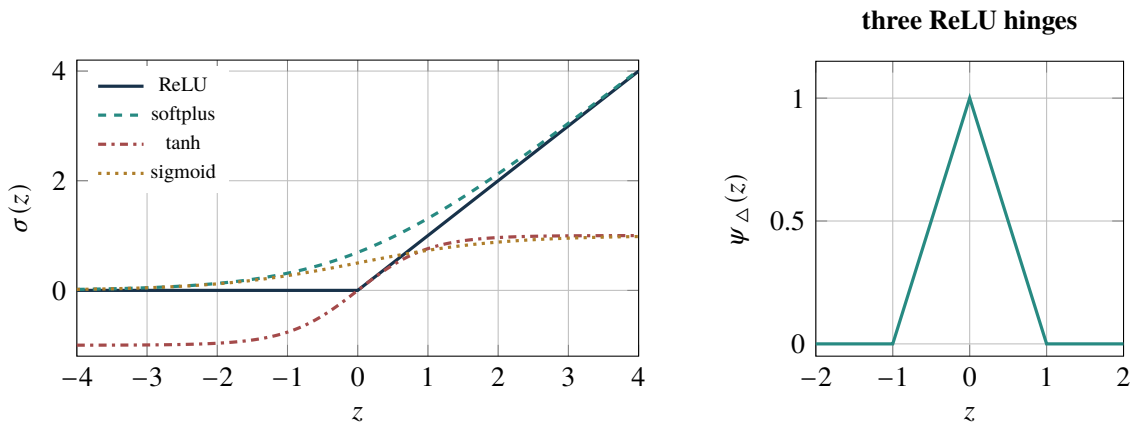


Figure 3.7: Activation functions and the localized triangular feature constructed from three ReLU hinges in Equation (3.25).

Encoding and representation capacity.

Automatic feature construction does not imply that raw measurements should always be supplied without structure. Continuous variables should be scaled; angles can be encoded by sine and cosine; discrete modes may use indicator or embedding vectors; and physically meaningful quantities such as relative speed, time headway, or constraint distance can be concatenated with learned features. A useful network may therefore combine direct feature extraction with representation learning.

The classical universal-approximation result of Cybenko (1989) states, under its activation and domain conditions, that sufficiently wide shallow networks can approximate continuous functions on compact sets arbitrarily well. This is a capacity statement, not a training or control guarantee. It does not specify the required width, ensure that finite-data optimization finds the approximation, or establish extrapolation, closed-loop performance, constraint satisfaction, or stability. Architecture and validation remain engineering decisions even when the function class is theoretically expressive enough.

3.3.2 Backpropagation and Parameter Updates

For a declared regression data set $\{(x^{(s)}, \beta^{(s)}, \omega_s)\}_{s=1}^q$, a common fixed-target objective for $\tilde{h}(x; \theta)$ is

$$F(\theta) = \frac{1}{q} \sum_{s=1}^q \ell_s(\theta) + \lambda_{\text{reg}} \mathcal{R}(\theta), \quad \ell_s(\theta) = \omega_s \|\tilde{h}(x^{(s)}; \theta) - \beta^{(s)}\|_2^2, \quad \omega_s \geq 0. \quad (3.26)$$

Here $\mathcal{R}$ is a declared regularizer, such as a coefficient or smoothness penalty, and $\lambda_{\text{reg}} \geq 0$ controls its strength. The empirical sum fits the sampled states; it does not imply equal accuracy over the entire state space. For a linear architecture this has the form of Equation (3.20); the factor $1/q$ only rescales the corresponding regularization coefficient. For an MLP the objective is generally nonconvex.

For finite-action classification, cross-entropy commonly replaces squared error:

$$\ell_s^{\text{CE}}(\theta) = -\omega_s \sum_{j=1}^{n_a} y_j^{(s)} \log \pi_{\theta}(u^{(j)} | x^{(s)}), \quad (3.27)$$

where $y^{(s)}$ is the one-hot target label. Thus the output semantics from Section 3.1.2 determine the loss as well as the last layer.

A scalar chain-rule calculation.

Before writing the multilayer recursion, consider one scalar input, one hidden unit, and one scalar output:

$$a = wx + b, \quad h = \sigma(a), \quad \tilde{y} = rh + b_o, \quad \ell = (\tilde{y} - \beta)^2. \quad (3.28)$$

Let $c = \partial\ell/\partial\tilde{y} = 2(\tilde{y} - \beta)$. Applying the chain rule from the output back toward the input gives

$$\frac{\partial\ell}{\partial r} = ch, \quad \frac{\partial\ell}{\partial b_o} = c, \quad (3.29)$$

$$\delta := \frac{\partial\ell}{\partial a} = cr\sigma'(a), \quad \frac{\partial\ell}{\partial w} = \delta x, \quad \frac{\partial\ell}{\partial b} = \delta. \quad (3.30)$$

The factor c is computed once and reused; the hidden error δ is then reused for both w and b . Differentiating every parameter independently would repeat these same products. Figure 3.8 summarizes this forward computation and backward reuse.

Backpropagation applies this reuse layer by layer. It computes the gradient of ℓ_s efficiently by storing the forward quantities and propagating derivatives from the output toward the input. For the scalar network in Equations (3.22) and (3.23), let $c_s = \partial\ell_s/\partial\tilde{J}$ and $\delta_\ell = \partial\ell_s/\partial a_\ell$. The backward recursion is

$$\delta_L = (r_o c_s) \odot \sigma'_L(a_L), \quad (3.31)$$

$$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot \sigma'_\ell(a_\ell), \quad \ell = L-1, \dots, 1, \quad (3.32)$$

$$\nabla_{W_\ell} \ell_s = \delta_\ell h_{\ell-1}^\top, \quad \nabla_{b_\ell} \ell_s = \delta_\ell, \quad (3.33)$$

$$\nabla_{r_o} \ell_s = c_s h_L, \quad \frac{\partial\ell_s}{\partial b_o} = c_s. \quad (3.34)$$

For the squared-error loss in Equation (3.26), $c_s = 2\omega_s(\tilde{J}(x^{(s)}; \theta) - \beta^{(s)})$; another loss changes this output derivative but not the layer-by-layer reuse. Equations (3.31)–(3.34) assemble the pathwise derivatives into the one-sample gradient $\nabla_\theta \ell_s$. The update rule below either uses that gradient directly or averages it over a declared collection of samples. For a nondifferentiable activation such as ReLU, its derivative at the kink is replaced by the implementation's declared subgradient convention. The first line initializes the hidden-layer recursion. Backpropagation is therefore a procedure for computing gradients, not an optimizer; an update rule still has to decide how samples are selected and how the gradient is scaled (Rumelhart et al. 1986).

All first-order variants can be written as

$$\theta_{i+1} = \theta_i - \eta_i d_i, \quad \eta_i > 0, \quad (3.35)$$

where i is the optimizer iteration and d_i is the chosen gradient estimate, not a new trainable

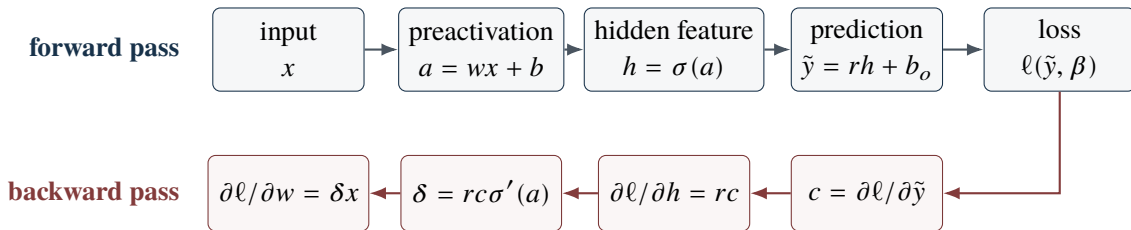


Figure 3.8: Forward evaluation and backward reuse for the scalar chain-rule calculation in Equations (3.28) to (3.30).

parameter. The most common choices are

$$d_i^{\text{full}} = \frac{1}{q} \sum_{s=1}^q \nabla \ell_s(\theta_i) + \lambda_{\text{reg}} \nabla \mathcal{R}(\theta_i), \quad (3.36)$$

$$d_i^{\text{one}} = \nabla \ell_{s_i}(\theta_i) + \lambda_{\text{reg}} \nabla \mathcal{R}(\theta_i), \quad (3.37)$$

$$d_i^{\text{mini}} = \frac{1}{|\mathcal{B}_i|} \sum_{s \in \mathcal{B}_i} \nabla \ell_s(\theta_i) + \lambda_{\text{reg}} \nabla \mathcal{R}(\theta_i). \quad (3.38)$$

The full-batch direction d_i^{full} uses all q samples at every update. By contrast, the one-sample direction d_i^{one} uses only the index s_i . A cyclic incremental method sets $s_i = 1 + ((i - 1) \bmod q)$, whereas classical stochastic gradient descent (SGD) draws s_i independently, usually uniformly, making the data term an unbiased finite-sum gradient estimate. Random reshuffling instead draws one permutation per epoch and visits every sample once; it is stochastic but differs from with-replacement SGD. Minibatch SGD averages over $\mathcal{B}_i$, reducing gradient variability and exploiting vectorized computation.

Far from a solution, inexpensive component gradients can make rapid progress. Near a solution, gradient noise or a repeating order can produce oscillation; minibatching, reshuffling, and stepsize reduction can mitigate it. The stepsize, batch rule, and stopping criterion must therefore be reported with the optimizer name.

Adaptive moment estimation (Adam) rescales a chosen gradient direction using running first- and second-moment estimates. Given an initialized parameter vector θ_1 , Adam sets $m_0 = v_0 = 0$ and, from optimizer iteration $i = 1$ onward, adds exponential first- and second-moment estimates (Kingma and Ba 2015):

$$m_i = \beta_1 m_{i-1} + (1 - \beta_1) d_i, \quad v_i = \beta_2 v_{i-1} + (1 - \beta_2) (d_i \odot d_i), \quad (3.39)$$

$$\hat{m}_i = \frac{m_i}{1 - \beta_1^i}, \quad \hat{v}_i = \frac{v_i}{1 - \beta_2^i}, \quad (3.40)$$

$$\theta_{i+1} = \theta_i - \eta_i \frac{\hat{m}_i}{\sqrt{\hat{v}_i + \delta_A}}. \quad (3.41)$$

Here $0 \leq \beta_1, \beta_2 < 1$, $\delta_A > 0$, and the arithmetic in the final line is componentwise. More generally, a positive diagonal scaling D_i gives $\theta_{i+1} = \theta_i - \eta_i D_i d_i$. Such scaling can compensate for unequal parameter sensitivities; Newton and Gauss–Newton methods replace it by a curvature-based linear solve. These choices may accelerate training but do not repair biased targets or guarantee a global solution of a nonconvex problem. When decoupled weight decay is intended with Adam, it should be stated explicitly rather than being conflated with an ℓ_2 loss penalty (Loshchilov and Hutter 2019).

Table 3.3 aligns the formulas with their data-selection rules. The full-batch row uses d_i^{full} , the next three rows give alternative ways to select the index in d_i^{one} , and the minibatch row uses d_i^{mini} . Adam is orthogonal to that sampling choice: it rescales whichever d_i has been selected, although minibatches are its most common use in neural-network training.

Table 3.3: Common first-order training choices. Adam can rescale any gradient estimate d_i , although it is most often paired with minibatches.

Method	Data used per update	Main strength	Main caution
Full batch , d_i^{full}	All q samples	Exact finite-sum direction	Expensive for large data sets
Cyclic incremental , d_i^{one}	One sample in fixed order	Inexpensive and deterministic	Order-dependent; can cycle near a solution
IID SGD , d_i^{one}	One sample with replacement	Unbiased data-gradient estimate	Noisy; may revisit or skip samples in an epoch
Random reshuffling , d_i^{one}	One sample per update; all q once per epoch	Complete epoch coverage	Within-epoch directions are dependent
Minibatch SGD , d_i^{mini}	Average over $\mathcal{B}_i$	Vectorization and lower variance	More work and memory per update
Adam	Any d_i ; commonly a minibatch direction	Adapts unequal coordinate scales	Extra state and hyperparameters

3.3.3 Structured Learned Encoders

An MLP treats its input as a fixed-dimensional vector with no built-in notion of neighborhood, ordering, or interaction topology. Mobility observations often have exactly such structure. A structured encoder imposes an *inductive bias*—a reusable assumption about which inputs share parameters or exchange information—and supplies its feature vector to a cost, Q-factor, policy, or model head. The target object and training loss remain separate choices. Writing the structured observation as $\mathcal{O}$, the common pattern is

$$\tilde{h}(\mathcal{O}; \theta) = g_\psi(e_\vartheta(\mathcal{O})), \quad \theta = (\vartheta, \psi), \quad (3.42)$$

where the encoder e_ϑ constructs a feature representation and the head g_ψ produces the required cost, Q-factor, policy, or model output.

The principal alternatives differ mainly in the structure assumed by the encoder e_ϑ :

Convolutional neural network (CNN).

Shared local filters suit images, occupancy grids, spatial sensor arrays, and regularly sampled signals; distant interactions may require several layers or an additional global mechanism.

Recurrent neural network (RNN).

A recurrent hidden state summarizes an ordered history. Gated variants such as long short-term memory (LSTM) are useful for streams and partial observability, but their sequential computation limits parallelism.

Graph neural network (GNN).

Shared message updates follow vehicle–vehicle or vehicle–infrastructure edges, accommodating variable numbers of agents; performance depends on how the graph and message depth are chosen.

Transformer.

Attention creates content-dependent interactions among time, agent, or map tokens and supports parallel training; positional or relational information must be supplied, and dense attention becomes costly for long sequences.

Learned state-space model (SSM) encoder.

A structured latent recurrence scales efficiently to long ordered sequences. Its latent state is a learned feature and should not automatically be interpreted as the physical system state.

These encoders are alternatives to, or components of, an MLP rather than new learning objectives. Hybrid designs are common, and each can feed the same output head and be trained with the losses and gradient rules developed above. Accordingly, an architecture name should be reported together with its actual inputs, ordering or graph, output head, and training objective.

3.4 Data, Regularization, and Validation

This section considers *fixed-target regression*: the targets have already been produced, and they remain fixed while a parameterized function is fitted. They may come from DP, simulation, measurements, or an approximate numerical calculation. In Chapter 4, Bellman, Monte Carlo, or temporal-difference updates repeatedly refresh the targets, introducing a coupled learning iteration that is outside the present analysis. Even with a fixed target, three questions remain: where accuracy is measured, which fitted solution is selected, and whether error entered before or during fitting. The least-squares viewpoint follows Bertsekas (2025, Sec. 3.1); the distribution and error-lineage organization below makes its control use explicit.

3.4.1 The Fitting Distribution Defines the Error Being Minimized

Let ρ be a probability distribution over states. The distribution- weighted error is

$$\mathcal{E}_\rho(\tilde{J}, J) = \mathbb{E}_{x \sim \rho} [(\tilde{J}(x) - J(x))^2]. \quad (3.43)$$

If states are sampled independently from ρ , the empirical mean square estimates the right-hand side. The distribution is therefore not merely a data source; it specifies which parts of the state space receive weight in the fitting objective. A nominal-driving distribution may place little mass near a collision set, friction limit, or actuator saturation. Small nominal error can therefore coexist with large error in rare but important regions.

A constant-fit example.

Let $X \sim \rho$, let $J(X) = X^2$, and restrict the approximation to the constant $\tilde{J}(X; r) = r$. Its best parameter is defined by

$$r_\rho^* := \arg \min_{r \in \mathbb{R}} \mathbb{E}_\rho[(r - X^2)^2]. \quad (3.44)$$

Differentiating the objective with respect to r gives $r_\rho^* = \mathbb{E}_\rho[X^2]$. Hence, for $X \sim \text{Unif}[-1, 1]$, $r_\rho^* = 1/3$; for $X \sim \text{Unif}[-1/2, 1/2]$, $r_\rho^* = 1/12$. Neither constant is intrinsically correct everywhere. Each is the best member of the declared architecture under a different fitting distribution. Richer linear and neural approximators inherit the same dependence.

Normalize continuous inputs before using polynomial, radial, or neural features. Otherwise physical units can dominate the conditioning of $\Phi^\top \Phi$ or the gradient scale. Normalization is an internal numerical transformation; reported states and control metrics should retain their physical units.

Let ρ_{train} , ρ_{val} , and ρ_{op} denote the training, validation, and closed-loop operating distributions. Errors under these distributions need not agree. Consecutive rows from one trajectory are also strongly correlated, so a random row split can place nearly duplicate states in training and validation sets. Prefer splits by complete trajectory, route, driver, controller, operating condition, or random seed, and include tests under declared shifts in mass, friction, delay, reference, or disturbance statistics.

3.4.2 What Regularization Selects

For labels $\beta^{(s)}$ and predictions $\tilde{\beta}(x^{(s)}; \theta)$, a regularized fit has the form

$$\min_{\theta} \frac{1}{q} \sum_{s=1}^q (\tilde{\beta}(x^{(s)}; \theta) - \beta^{(s)})^2 + \lambda_{\text{reg}} \mathcal{R}(\theta), \quad \lambda_{\text{reg}} \geq 0. \quad (3.45)$$

The coefficient λ_{reg} trades agreement with the supplied labels against the declared preference $\mathcal{R}$. Regularization is useful when several parameter vectors fit the data similarly or the unregularized fit is poorly conditioned, but it deliberately changes the solution and can introduce bias. Representative explicit penalties are

$$\mathcal{R}_2(\theta) = \frac{1}{2} \|\theta - \theta_{\text{ref}}\|_2^2 = \frac{1}{2} \sum_j (\theta_j - \theta_{\text{ref},j})^2, \quad (3.46)$$

$$\mathcal{R}_1(\theta) = \|\theta\|_1 = \sum_j |\theta_j|, \quad (3.47)$$

$$\mathcal{R}_{\text{smooth}}(\theta) = \|L_{\text{reg}} \theta\|_2^2. \quad (3.48)$$

Here $\|z\|_2 = (\sum_j z_j^2)^{1/2}$ is the Euclidean, or ℓ_2 , norm, whereas $\|z\|_1 = \sum_j |z_j|$ is the ℓ_1 norm. The ℓ_2 penalty shrinks the parameters toward the nominal parameter θ_{ref} and can improve conditioning, but excessive shrinkage can suppress meaningful departures from that nominal value. The ℓ_1 penalty promotes sparse coefficients; with strongly correlated features, however, which coefficient is retained can be unstable. In the smoothness penalty, L_{reg} measures neighboring differences, discrete derivatives, or another declared notion of roughness; it can suppress oscillation while

also smoothing a genuine sharp transition. Whether biases or intercepts are excluded from these penalties must be stated.

Early stopping and dropout regularize through the training procedure rather than an explicit additive penalty (Srivastava et al. 2014). Early stopping retains a validation-selected iterate before fully minimizing the training loss; dropout trains with randomly omitted activations and uses the complete network at inference. Decoupled weight decay with an adaptive optimizer is not generally identical to adding an ℓ_2 term to the loss (Loshchilov and Hutter 2019).

Regularization can improve conditioning and shifted validation performance, but it deliberately introduces bias. It cannot correct a biased DP target, an inaccurate model, or an unrepresentative data distribution. A reproducible report should therefore state the mechanism, coefficient or stopping rule, parameters excluded from the penalty, and validation distribution.

Figure 3.9 holds the noisy samples and RBF architecture fixed and changes only the selection rule. The four curves show why “regularization” is not a single effect: coefficient shrinkage, sparsity, and smooth variation encode different preferences.

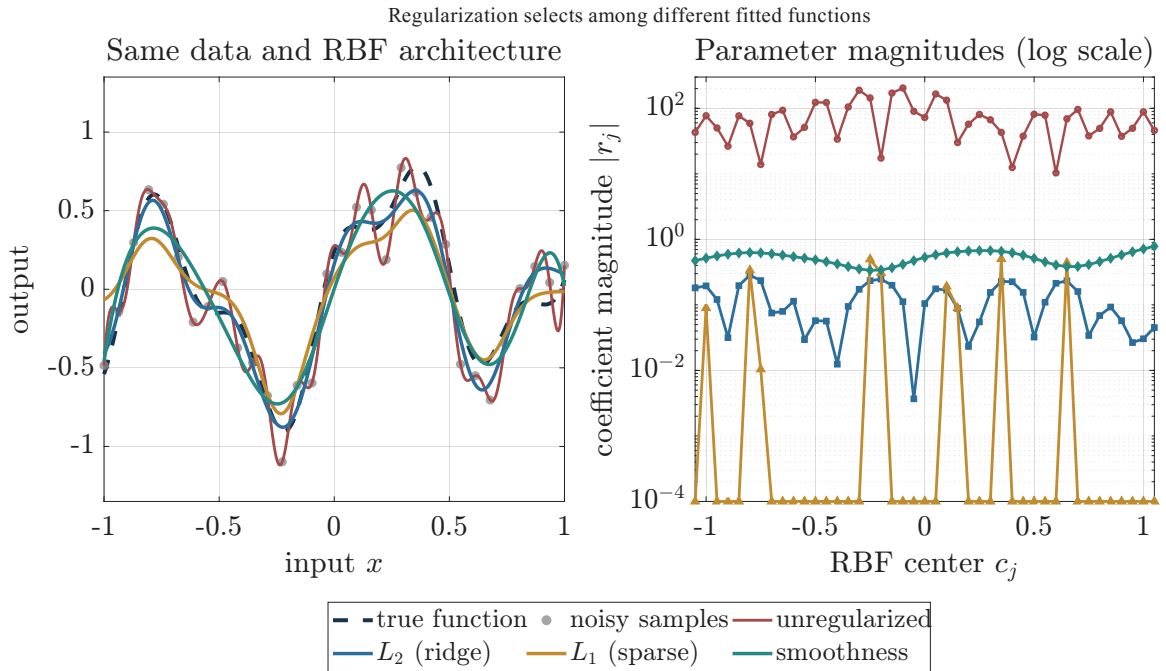


Figure 3.9: Illustrative effects of regularization in fixed-RBF regression. The unregularized fit follows sample noise using large coefficients; ℓ_2 , ℓ_1 , and smoothness penalties instead favor coefficient shrinkage, sparsity, and gradual coefficient variation. Exact zeros in the logarithmic coefficient plot are shown at the numerical floor.

3.4.3 Noisy Targets and Error Lineage

Suppose a scalar label satisfies

$$\beta = m(x) + \varepsilon, \quad m(x) = \mathbb{E}[\beta | x], \quad \mathbb{E}[\varepsilon | x] = 0. \quad (3.49)$$

For any fixed prediction a ,

$$\mathbb{E}[(a - \beta)^2 | x] = (a - m(x))^2 + \text{Var}[\beta | x]. \quad (3.50)$$

The cross term vanishes because $\mathbb{E}[\beta - m(x) | x] = 0$. Hence unrestricted squared-error regression targets the conditional mean, whereas a restricted architecture finds the member of its class that best approximates that mean under the fitting distribution. For M conditionally independent and identically distributed labels at the same state,

$$\bar{\beta}(x) = \frac{1}{M} \sum_{j=1}^M \beta_j(x), \quad \text{Var}[\bar{\beta}(x) | x] = \frac{1}{M} \text{Var}[\beta(x) | x]. \quad (3.51)$$

Thus $M = 25$ independent repeats reduce conditional variance by a factor of 25 and conditional standard deviation by a factor of five. The reduction does not hold unchanged for correlated rollouts that reuse the same disturbance realization, initial trajectory, or model error.

More fundamentally, fitting an available target $\hat{J}$ is different from approximating the desired object J . Define $\|v\|_\rho := (\mathbb{E}_{x \sim \rho}[v(x)^2])^{1/2}$; then

$$\tilde{J} - J = (\tilde{J} - \hat{J}) + (\hat{J} - J), \quad \|\tilde{J} - J\|_\rho \leq \|\tilde{J} - \hat{J}\|_\rho + \|\hat{J} - J\|_\rho. \quad (3.52)$$

The first term contains representation, finite-data, regularization, and optimization effects. The second contains model, discretization, sampling, or target-generation error. Figure 3.10 makes the two links explicit: validation against the available target examines only the lower link.

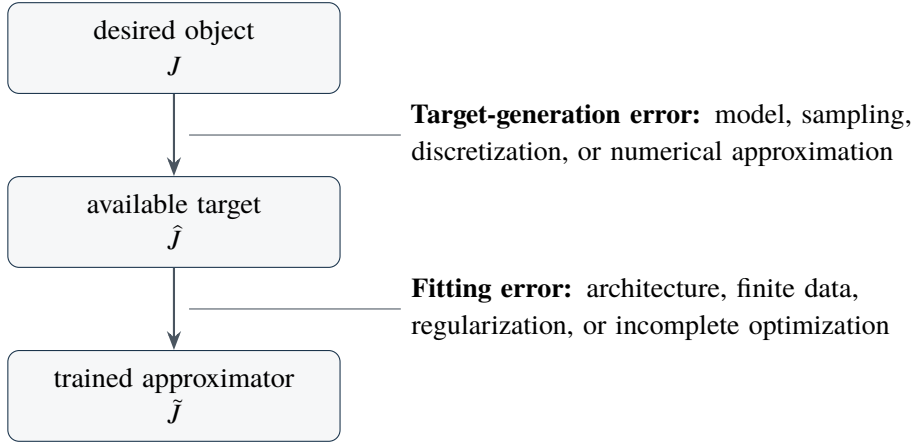


Figure 3.10: Error lineage. Validation only against $\hat{J}$ examines the lower link and cannot detect bias already present in the supplied target.

For example, a network can interpolate a coarse-grid DP calculation perfectly and still disagree with the continuous physical problem. Independent reference calculations, Bellman-residual checks, shifted tests, and ultimately closed-loop evaluation address different links in this chain.

3.4.4 Control-Relevant Validation

Prediction error alone does not determine whether an approximator produces a useful control decision. For a nonempty finite action set $\mathcal{U}(x)$, let Q^* be the reference Q-factor and $\tilde{Q}$ its approximation. Define

$$u^*(x) \in \arg \min_{u \in \mathcal{U}(x)} Q^*(x, u), \quad \hat{u}(x) \in \arg \min_{u \in \mathcal{U}(x)} \tilde{Q}(x, u), \quad (3.53)$$

$$\mathcal{R}_{\text{dec}}(x) = Q^*(x, \hat{u}(x)) - Q^*(x, u^*(x)) \geq 0. \quad (3.54)$$

Decision regret evaluates both actions with the same reference Q-factor. It is neither a closed-loop excess-cost bound nor the cumulative regret used in online learning. The same comparison applies when a fitted cost-to-go function is inserted into a one-step lookahead: its error matters only insofar as it changes the relative costs of candidate actions. Thus a constant offset can produce nonzero prediction error while preserving the greedy action, whereas a smaller action-dependent error can change the decision. For continuous actions, the analogous concern is the error slope and curvature near a minimizer. Prediction RMSE, action agreement, one-step decision regret, and closed-loop cost therefore answer different validation questions.

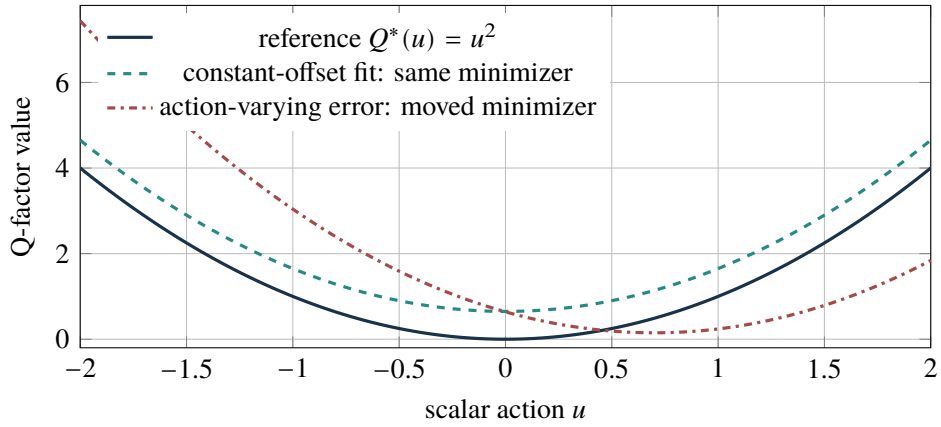


Figure 3.11: Prediction error matters through its variation with the action. A constant offset leaves the greedy action unchanged, whereas an action-dependent error can move the minimizer.

3.5 Mobility Case: A Car-Following Cost Surrogate

The common experiment in this section fits a cost-to-go function in a small longitudinal-control problem. Its role is to make the target, architecture, data support, and induced action auditable; it is not a high-fidelity car-following benchmark. It also differs from the finite-state examples of Chapter 2: the state is continuous, the five controls are discrete, and a grid plus interpolation supplies a numerical DP reference.

3.5.1 Declared Model and Numerical Target

Let $e_d = d - d_{\text{ref}}$ be spacing error and $e_v = v_{\text{lead}} - v_{\text{ego}}$ be relative speed. Thus $e_v < 0$ means that the ego vehicle is closing on the lead vehicle. With a constant-speed lead vehicle and zero-order-held ego acceleration u ,

$$e_{d,k+1} = e_{d,k} + T_s e_{v,k} - \frac{1}{2} T_s^2 u_k, \quad (3.55)$$

$$e_{v,k+1} = e_{v,k} - T_s u_k. \quad (3.56)$$

Figure 3.12 fixes the sign convention. A positive u accelerates the ego vehicle, reduces e_v , and changes the spacing over the next sample.

The horizon is $N = 20$, $T_s = 0.25$ s, and

$$u_k \in \mathcal{U}_{\text{cf}} := \{-3, -1.5, 0, 1.5, 3\} \text{ m/s}^2. \quad (3.57)$$

For $d_{\text{ref}} = 15$ m and $d_{\text{safe}} = 5$ m, use

$$g(e_d, e_v, u) = 0.05e_d^2 + 0.30e_v^2 + 0.035u^2 + 12[d_{\text{safe}} - (d_{\text{ref}} + e_d)]_+^2. \quad (3.58)$$

The terminal cost is

$$G(e_d, e_v) = 0.8e_d^2 + 1.5e_v^2 + 40[d_{\text{safe}} - (d_{\text{ref}} + e_d)]_+^2. \quad (3.59)$$

Backward DP is run on 61 spacing-error points over $[-12, 12]$ m, 49 relative-speed points over $[-6, 6]$ m/s, and the five controls in $\mathcal{U}_{\text{cf}}$. At a grid state, each action produces a generally off-grid successor. The stored cost at that successor is therefore evaluated by *bilinear interpolation*: the four surrounding grid values are weighted according to the successor's relative horizontal and vertical position. This is the two-dimensional analogue of linear interpolation.

Interpolation is defined only inside the rectangular grid. For a raw successor (e_d^+, e_v^+) , let $\bar{e}_d^+ = \text{clip}_{[-12, 12]}(e_d^+)$ and $\bar{e}_v^+ = \text{clip}_{[-6, 6]}(e_v^+)$. Bilinear interpolation uses $(\bar{e}_d^+, \bar{e}_v^+)$ and adds the

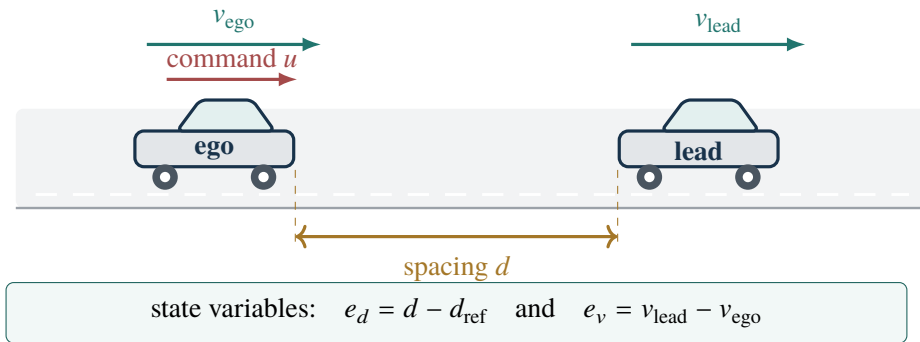


Figure 3.12: Variables in the car-following teaching model. The reference spacing d_{ref} converts physical spacing into the state e_d ; the second state e_v records whether the gap is opening or closing.

numerical boundary penalty $35[(e_d^+ - \bar{e}_d^+)^2 + (e_v^+ - \bar{e}_v^+)^2]$. Clipping only supplies an interpolation point; the penalty charges and thereby discourages departures from the grid instead of making boundary saturation artificially free. It is distinct from the soft physical safety penalty in Equation (3.58). These choices completely specify the reference numerical target J_1^{ref} ; it is a grid-based approximation, not an exact solution of the continuous-state problem. Neither the safety threshold nor the numerical grid boundary is imposed as a hard state constraint. Every action in $\mathcal{U}_{\text{cf}}$ remains admissible, but an action that approaches unsafe spacing or sends its successor outside the value grid incurs the corresponding finite penalty. By contrast, the finite set $\mathcal{U}_{\text{cf}}$ is a hard input constraint. The known model is used both to construct the target and to compare the actions induced by each fitted approximation.

3.5.2 Architectures and Data Support

Four architectures are fitted to the same J_1^{ref} :

1. **Quadratic fixed** uses normalized quadratic features $[1, z_d, z_v, z_d^2, z_d z_v, z_v^2]^\top$, where $z_d = e_d/12$ and $z_v = e_v/6$.
2. **Structured fixed** adds the proximity and closing features

$$h = \left[\frac{-8 - e_d}{4} \right]_+, \quad c_v = [-z_v]_+ = \left[-\frac{e_v}{6} \right]_+, \quad [h, h^2, hc_v, h^2 c_v]^\top. \quad (3.60)$$

Thus the added vector contains proximity terms and proximity–closing interaction terms; c_v is not used alone. The normalized quantity c_v is a dimensionless closing-speed hinge: it is positive only when $e_v < 0$, and normalization keeps its scale comparable with the other fitted features. The threshold $e_d = -8$ m activates the feature 2 m before the soft safety threshold $e_d = d_{\text{safe}} - d_{\text{ref}} = -10$ m. Neither threshold is a hard constraint.

3. **RBF fixed** uses an intercept, two affine terms, and 35 Gaussian features. In normalized coordinates $z_d = e_d/12$ and $z_v = e_v/6$, its centers are the 35 pairs from seven points in $[-0.9, 0.85]$ and five points in $[-0.8, 0.8]$, with common width 0.38. The total parameter dimension is therefore $1 + 2 + 35 = 38$.
4. **Shallow MLP** uses 24 tanh units and a linear scalar output, trained by minibatch Adam. Its two inputs are scaled by the declared grid half-ranges 12 and 6; only target centering and scaling use the training subset. Validation selects the retained checkpoint.

Write the coefficient vector as $r = (r_0, \bar{r}^\top)^\top$, where r_0 is the intercept and $\bar{r}$ contains all remaining coefficients. The three fixed-feature models use the standard ridge objective

$$\min_{r_0, \bar{r}} \|\Phi r - y_n\|_2^2 + \lambda \|\bar{r}\|_2^2,$$

where y_n collects the standardized training targets. Thus the ridge coefficient λ applies an ℓ_2 penalty to every nonintercept coefficient, while r_0 remains unpenalized. The Quadratic and Structured fixed fits use $\lambda = 10^{-5}$; the RBF fit uses $\lambda = 3 \times 10^{-4}$. These values multiply an unnormalized sum of squared errors, so they should not be compared numerically with a

coefficient attached to a mean loss. All models fit the same standardized target and are transformed back to cost units before evaluation. The declared training support is

$$-10.5 \leq e_d \leq 10, \quad |e_v| \leq 4.75. \quad (3.61)$$

Equation (3.61) defines a rectangular *fitting support* inside the full $61 \times 49 = 2989$ -state DP grid; it is not a physical state constraint. The rectangle contains 2028 grid states. A fixed random permutation assigns 1521 of them to training and splits the remaining 507 into 253 model-selection validation states and 254 disjoint in-support test states. These three subsets are interspersed throughout the same rectangle rather than occupying separate spatial regions. The MLP checkpoint is selected only with the validation subset, while the in-support test is reserved for reporting.

The other 961 grid states form the *outside-support test*. They are the deterministic boundary shell outside the rectangle, not another random sample from the fitting distribution. No outside-support state is used for fitting or model selection; the shell only probes extrapolation. This fixed grid split is also different from a trajectory-level generalization test. Figure 3.13 separates these roles visually. The red safety-threshold line is shown because it cuts through the fitting support, but it does not define any of the four data subsets.

Figure 3.14 then compares the target and all four fitted surfaces on their common state grid.

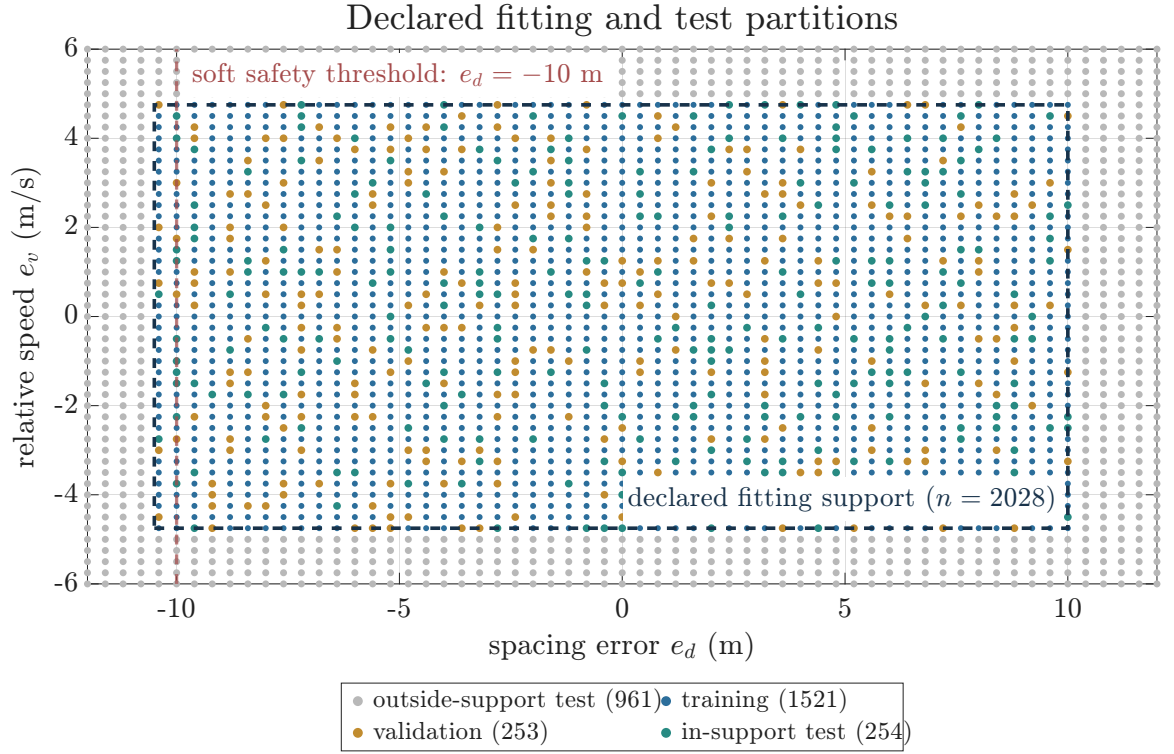


Figure 3.13: Declared data partitions on the common DP grid. The dashed rectangle is the fitting support from Equation (3.61); its states are randomly divided into training, model-selection validation, and held-out in-support test subsets. The surrounding grid shell is used only as an outside-support extrapolation test. The soft safety threshold is a separate property of the cost function.

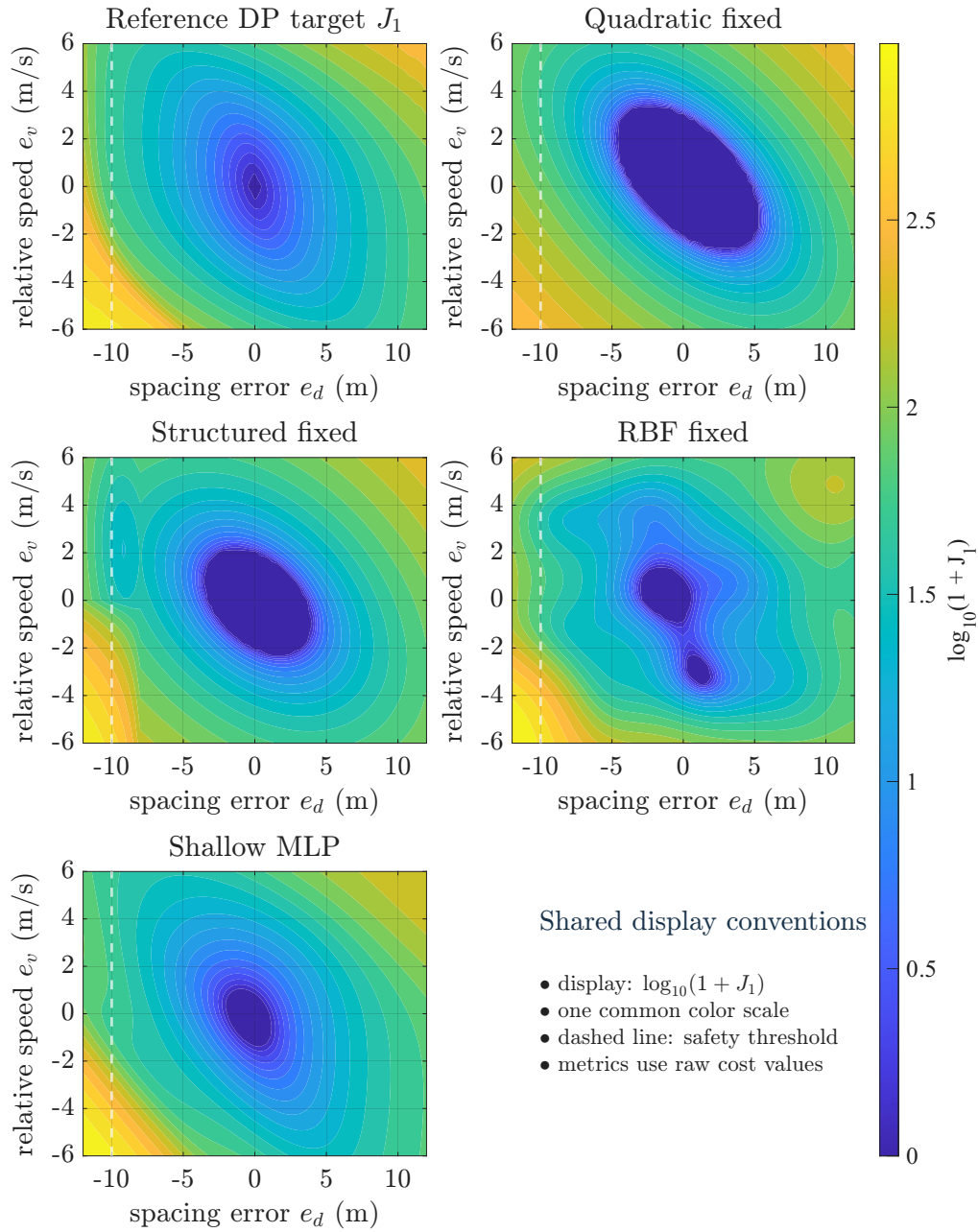


Figure 3.14: Car-following cost-to-go comparison. Five panels use the common $\log_{10}(1 + J_1)$ color scale for the reference target and four fitted architectures. The final panel summarizes the shared display conventions. Values below zero are displayed as zero and values above the reference maximum saturate the shared color scale; all reported metrics use the raw predictions. None of the fits reproduces the reference surface everywhere outside the declared training support.

3.5.3 Prediction Quality and Induced Decisions

Each fitted J_1 is inserted into the same one-step model-based minimization. Let $Q_0^{\text{ref}}(x, u)$ denote the stage-zero action cost obtained from g , J_1^{ref} , the same bilinear interpolation, and the same off-grid penalty. Write $f_{\text{clip}}(x, u) = (\bar{e}_d^+, \bar{e}_v^+)$ for the clipped successor and

$$c_{\text{bd}}(x, u) = 35 \left[(e_d^+ - \bar{e}_d^+)^2 + (e_v^+ - \bar{e}_v^+)^2 \right].$$

For architecture j , the fitted tail cost instead defines

$$\tilde{Q}_{0,j}(x, u) = g(x, u) + c_{\text{bd}}(x, u) + \tilde{J}_{1,j}(f_{\text{clip}}(x, u)), \quad (3.62)$$

$$\hat{u}_j(x) \in \arg \min_{u \in \mathcal{U}_{\text{cf}}} \tilde{Q}_{0,j}(x, u). \quad (3.63)$$

Inside the clipped continuous domain, the fitted function is evaluated directly; only the tabulated reference J_1^{ref} requires bilinear interpolation. Thus both calculations use the same declared clipping and boundary penalty. Because J_1 is the optimal tail cost from stage 1 onward, it is precisely the quantity needed to compare stage-zero actions. The experiment then evaluates

$$\mathcal{R}_{\text{dec},j}^{\text{ref}}(x) = Q_0^{\text{ref}}(x, \hat{u}_j(x)) - \min_{u \in \mathcal{U}_{\text{cf}}} Q_0^{\text{ref}}(x, u). \quad (3.64)$$

Prediction and action metrics use the same declared numerical reference. RMSE denotes root-mean-square cost error on the stated data subset; action agreement is the percentage of states whose selected action equals the reference action. Agreement is a label metric: it treats every mismatch equally and is sensitive to tie breaking. Decision regret has cost units and weights a mismatch by its reference cost consequence; it is zero for any truly optimal tied action. Exact ties in the implementation are resolved by the first action in the ordered set $\mathcal{U}_{\text{cf}}$.

Tables 3.4 and 3.5 report prediction, action-agreement, and decision-regret metrics separately.

Table 3.4: Fixed-seed Chapter 3 approximation results on two held-out test sets. *In-support* denotes the test split inside the declared training support; *outside-support* denotes the boundary-shell test.

Architecture	RMSE in	RMSE outside	Action match in	Action match outside
Quadratic fixed	27.994	116.415	83.1%	93.1%
Structured fixed	15.855	65.463	82.7%	89.9%
RBF fixed	6.521	61.713	63.8%	77.4%
Shallow MLP	3.918	41.069	86.2%	88.3%

Table 3.5: Mean one-step decision regret in reference cost units. Lower is better; zero means that the selected action is reference optimal.

Architecture	In-support test	Outside-support test
Quadratic fixed	0.0341	0.0257
Structured fixed	0.0454	0.0845
RBF fixed	0.7114	9.7711
Shallow MLP	0.0937	0.6909

Figure 3.15 visualizes the numerical entries in Tables 3.4 and 3.5: its first two panels reproduce the RMSE and action-agreement columns of Table 3.4, and its third panel reproduces Table 3.5.

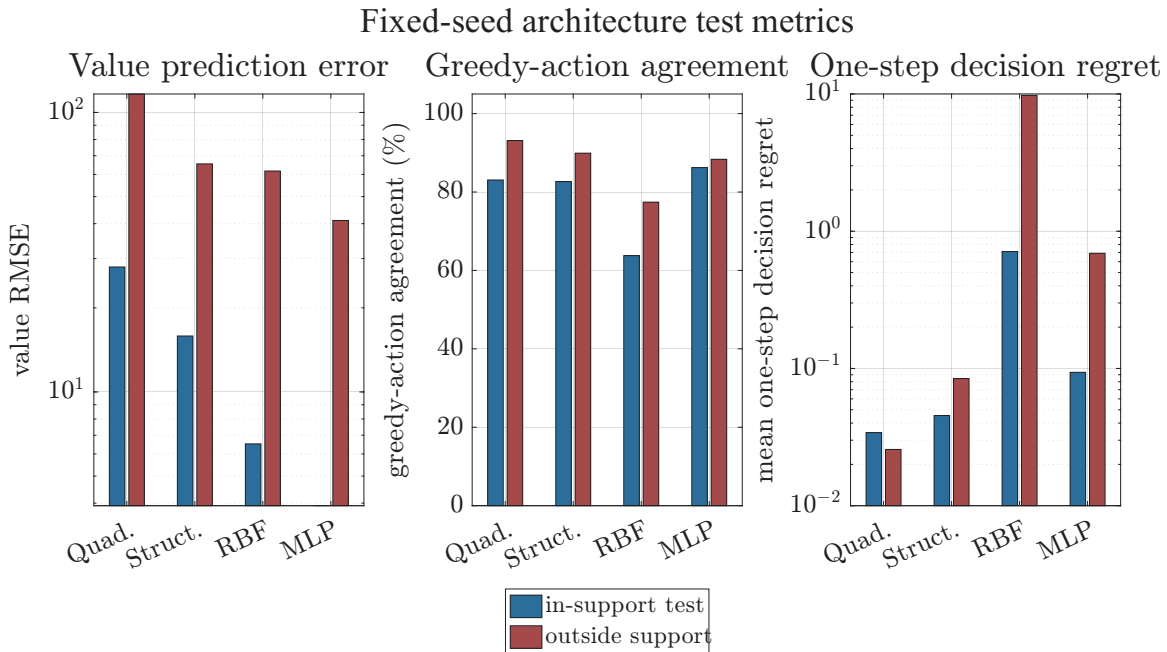


Figure 3.15: Prediction and control-facing metrics for the fixed-seed car-following experiment.

In support is the held-out in-support test, not the validation subset used to select the MLP checkpoint. RMSE, action agreement, and one-step decision regret measure different consequences of the same fitted cost-to-go. The plotted values are those reported in Tables 3.4 and 3.5.

The Shallow MLP fits the sampled cost-to-go surface best, but the Quadratic fixed model has the smallest outside-support decision regret in this run. The training objective minimizes cost-function MSE, not action-ordering error. Moreover, much of the reference surface is smooth and 900 of the 961 outside-support states (93.7%) use one of the two extreme reference actions.

A globally biased quadratic surface can therefore preserve the ordering of the five actions despite a large cost-to-go error. This is a fixed-seed illustration of metric mismatch, not a general ranking of shallow networks and fixed features. The same distinction explains the in-support metrics. Although the Quadratic fit produces more action mismatches than the MLP, its mismatched actions have a lower average regret: approximately 0.20, compared with 0.68 for the MLP. Thus, disagreement frequency alone does not reveal the cost consequence of the selected actions. Different neural initializations can change the MLP metrics.

Figure 3.16 makes the induced action ordering visible. The Quadratic fit preserves the broad diagonal bands of the reference map, whereas the MLP introduces localized action differences near the left side of the grid. These maps must be read together with the fitted surfaces in Figure 3.14: an action map retains only which candidate action has the smallest fitted cost, not the cost surface itself.

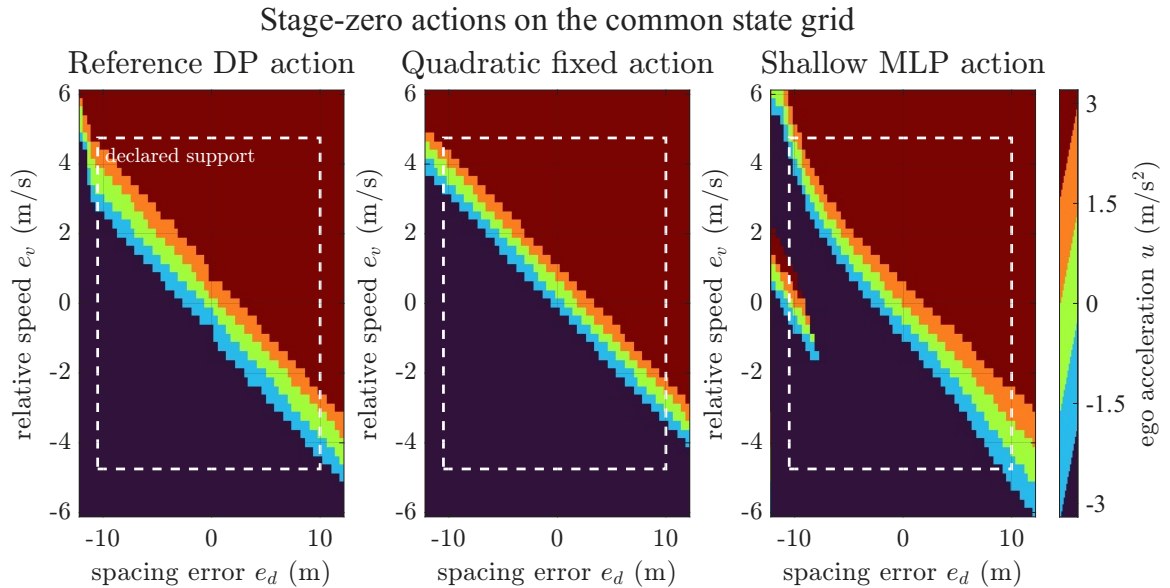


Figure 3.16: Reference, Quadratic fixed, and Shallow MLP stage-zero actions over the complete numerical grid. The dashed rectangle marks the declared training support; states outside it belong to the outside-support test. The architectures are shown as contrasting global and flexible approximators, not as a post-hoc ranking. Read with Figure 3.14, the broad diagonal bands show how markedly different fitted surfaces can preserve much of the same action ordering.

3.6 Summary

This chapter isolated representation and fixed-target fitting. Chapters 4, 6, and 7 embed the same architectures in value, policy, and model-learning iterations.

- The object and output semantics come first. Cost-to-go functions, Q-factors, and models typically use regression; finite-action policies may use classification; and continuous-action policies use vector regression. A stored approximator is not by itself a DP or RL algorithm.
- Linear-in-parameter architectures may use nonlinear fixed features and admit least-squares or ridge solutions. Neural architectures instead learn their features jointly through back-propagation and gradient-based updates; structured encoders provide alternative representations for spatial, graph, and sequential data.
- The fitting distribution and declared support determine where error is minimized. Training, model-selection validation, and test roles must remain separate. Regularization selects among fits but cannot repair a biased target or missing data support.
- Target-generation error and fitting error are distinct. Agreement with an approximate numerical or measured target does not establish agreement with the desired cost function, policy, or model.
- Prediction RMSE, greedy-action agreement, one-step decision regret, and closed-loop performance answer different questions. In the car-following case, the best cost fit did not necessarily have the smallest decision regret, particularly outside the fitting support.

Bibliography

- Bertsekas, D. P. (2025). *A Course in Reinforcement Learning*. 2nd ed. Nashua, NH: Athena Scientific. ISBN: 9781886529298.
- Cybenko, G. (1989). “Approximation by Superpositions of a Sigmoidal Function”. In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314. DOI: [10.1007/BF02551274](https://doi.org/10.1007/BF02551274). URL: <https://doi.org/10.1007/BF02551274>.
- Kingma, D. P. and J. Ba (2015). “Adam: A Method for Stochastic Optimization”. In: *International Conference on Learning Representations*. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980). URL: <https://arxiv.org/abs/1412.6980>.
- Loshchilov, I. and F. Hutter (2019). “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Rumelhart, D. E., G. E. Hinton, and R. J. Williams (1986). “Learning Representations by Back-Propagating Errors”. In: *Nature* 323, pp. 533–536. DOI: [10.1038/323533a0](https://doi.org/10.1038/323533a0). URL: <https://doi.org/10.1038/323533a0>.
- Srivastava, N., G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov (2014). “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.56, pp. 1929–1958. URL: <https://www.jmlr.org/papers/v15/srivastava14a.html>.